



Technical University of Cluj-Napoca
Faculty of Automation and Computer Science
Computer Science Department

Rolling Average of a Parallel 8-bit Data Stream

Puşcaş Alexandru

RÁCZ Dalma

Group 30411

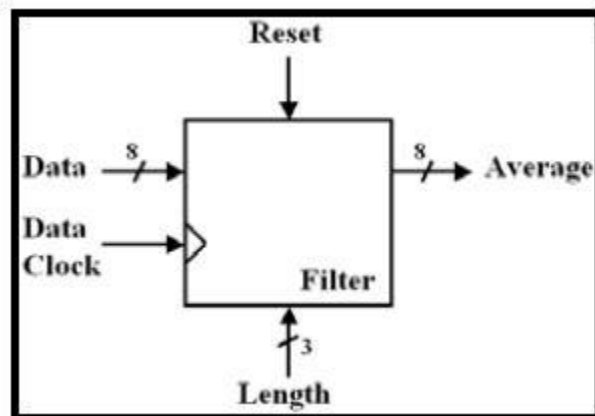
Table of contents

I. Task	3
II. Block Scheme.....	4
III. List of Components.....	4
IV. Significance of the Pins	5
V. Implementation	5
1.Pseudo Random Number Generator	5
2. Filter	8
3. Shift Register	13
4. Frequency Divider.....	14
5. Frequency Divider for the Anode	15
VI. Inputs. Outputs. Internal Signals	16
VII. Justification of the Chosen Solution.....	18
VIII. User's Manual	18
1. Active HDL.....	18
2. Xilinx ISE and the FPGA Board.....	19
IX. Possibilities of Further Development and Improvement.	21

I. Task

The Design Assignment will be to develop a simple signal processing system that will calculate the rolling average of a parallel 8-bit data stream as a systems design exercise. The design will be implemented on a self contained Xilinx/Digilent Spartan 3 XC3S200 FPGA board to allow demonstration of a working system. The system will be developed as a VHDL model using Xilinx ISE WebPack Version 6.3 that includes the use of the Modelsim simulation tools for design verification.

The task is to develop a VHDL based model for the "Digital Filter / Rolling Average" system combined with a data stream generator. The filter system will be required to run in "real time" and output the average value at the same rate as the original input data. Switches, Buttons and the Seven Segment Display located on the Diligent S3 board will need to be included to demonstrate correct operation.



Specifications for the switches:

Control Settings:

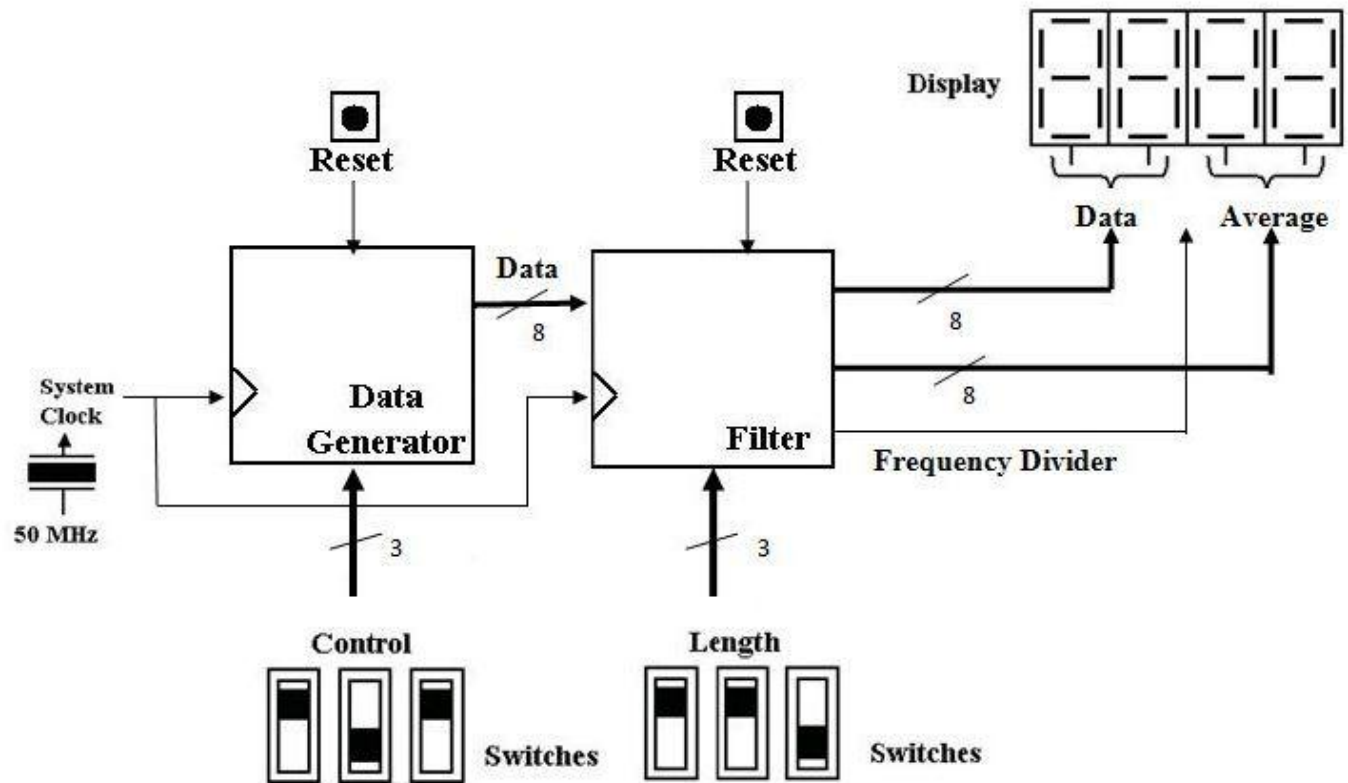
Off - Off - Off	Test Mode o/p 0 (Zero)
Off - Off - On	Square wave (0.25 x data clock)
Off - On - Off	Repeated 6 digit Sequence for Student Number One
Off - On - On	Repeated 6 digit Sequence for Student Number Two
On - On - Off	Pseudo Random Sequence reduced range 0 to 15
On - On - On	Pseudo Random Sequence full range 0 to 255

Buffer "Length" Settings:

Off - Off - Off	Stop - Hold Value
On - Off - Off	2 Sample Average
On - Off - On	4 Sample Average
On - On - Off	8 Sample Average
On - On - On	16 Sample Average

"Data Clock" rate: 1Hz.

II. Block Scheme



III. List of Components

The components which form the system are:

- pseudo random number generator (depending on the Control selections, it generates 2 different 6 digit sequences and pseudo random sequences in the range 0-15 or 0-255, again depending on the Control selections)
- filter
- shift register
- frequency divider – one for the clock (1 second) and another one for the display (to be able to display on all four displays different elements)

IV. Significance of the Pins

The interface with the user is done with the help of the switches. By means of the control switches, the user can choose, using different combinations, from one of the following options: Test, Pseudo random numbers in the range 0-15 or Pseudo random numbers in the range 0-255.

Using the control switches for length, the user can choose, again using different combinations, from one of the following options: Stop – Hold Value, 2 Sample Average, 4 Sample Average, 8 Sample Average or 16 Sample Average.

Two buttons will be used for Reset, one for the Control Switches and another one for the Length Switches.

V. Implementation

The components used for the implementation are the following ones: pseudo random number generator, filter, shift register, frequency divider and frequency divider for the anode. The VHDL code for these components is presented below:

1. Pseudo Random Number Generator

The two repeated 6 digit sequences are the following: (1,2,3,4,5,6) for student1 and (9,8,7,6,5,4) for student2. The sequence generated for the 4 bit numbers is F,E,C,8,1,2,4,9,3,6,D,A,5,B,7. For the 8 bit option there are 64 numbers generated, starting from FF.

```
library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity DataGenerator is
    port ( Clk, Reset : in STD_LOGIC;
          Control : in STD_LOGIC_VECTOR (2 downto 0);
          Q : inout STD_LOGIC_VECTOR (7 downto 0));
end entity;

architecture Attempting2 of DataGenerator is
```

```

signal Slow: STD_LOGIC;

component FrequencyDivider is
    port(CLK, Reset : in STD_LOGIC;
          Q : out STD_LOGIC);
end component;

begin
    div: FrequencyDivider port map (CLK => Clk, Reset => '1', Q => Slow);
    process(Slow, Control)
        type StudentArray is array (0 to 5) of STD_LOGIC_VECTOR (7 downto 0);
        variable D7, D6, D5, D4, D3, D2, D1, D0: STD_LOGIC := '1';
        variable DD3, DD2, DD1, DD0: STD_LOGIC := '1';
        variable Q7, Q6, Q5, Q4, Q3, Q2, Q1, Q0: STD_LOGIC := '0';
        variable QQ3, QQ2, QQ1, QQ0: STD_LOGIC := '0';
        variable i : integer range 0 to 5 := 0;
        variable k : integer range 0 to 5 := 0;
        variable n : integer range 0 to 3 := 0;
        variable x : STD_LOGIC := '0';
        variable Student1 : StudentArray := (0 => "00000001", 1 => "00000010", 2 =>
"00000011", 3 => "00000100", 4 => "00000101", 5 => "00000110" );
        variable Student2 : StudentArray := (0 => "00001001", 1 => "00001000", 2 =>
"00000111", 3 => "00000110", 4 => "00000101", 5 => "00000100" );
        begin
            if reset = '1' then
                q <= "00000000";
            else
                if (Control(2) = '1') and (Control(1) = '1') and (Control(0) = '0') then
                    if Rising_Edge(Slow) then
                        QQ0 := DD0;
                        QQ1 := DD1;
                        QQ2 := DD2;
                        QQ3 := DD3;
                        DD0 := QQ2 xor QQ3;
                        DD1 := QQ0;
                        DD2 := QQ1;
                        DD3 := QQ2;
                    end if;
                    Q(0) <= QQ0;
                    Q(1) <= QQ1;
                    Q(2) <= QQ2;
                    Q(3) <= QQ3;
                    Q(4) <= '0';
                    Q(5) <= '0';
                    Q(6) <= '0';
                    Q(7) <= '0';
                end if;
            end if;
        end process;
    end begin;

```

```

elsif (Control(2) = '1') and (Control(1) = '1') and (Control(0) = '1') then
    if rising_edge(Slow) then
        Q0 := D0;
        Q1 := D1;
        Q2 := D2;
        Q3 := D3;
        Q4 := D4;
        Q5 := D5;
        Q6 := D6;
        Q7 := D7;
        D0 := Q6 xor Q7;
        D1 := Q0;
        D2 := Q1;
        D3 := Q2;
        D4 := Q3;
        D5 := Q4;
        D6 := Q5;
        D7 := Q6;
    end if;
    Q(0) <= Q0;
    Q(1) <= Q1;
    Q(2) <= Q2;
    Q(3) <= Q3;
    Q(4) <= Q4;
    Q(5) <= Q5;
    Q(6) <= Q6;
    Q(7) <= Q7;
elsif (Control(2) = '0') and (Control(1) = '0') and (Control(0) = '0') then
    if (Slow'Event) and (Slow = '1') then
        Q(0) <= '0';
        Q(1) <= '0';
        Q(2) <= '0';
        Q(3) <= '0';
        Q(4) <= '0';
        Q(5) <= '0';
        Q(6) <= '0';
        Q(7) <= '0';
    end if;
elsif (Control(2) = '0') and (Control(1) = '1') and (Control(0) = '0') then
    if rising_edge(Slow) then
        i := i + 1;
        if (i > 5) then
            i := 0;
        end if;
    end if;
end if;
Q <= Student1(i);

```

```

        elsif (Control(2) = '0') and (Control(1) = '1') and (Control(0) = '1') then
            if rising_edge(Slow) then
                k := k + 1;
                if (k > 5) then
                    k := 0;
                end if;
            end if;
            Q <= Student2(k);

        elsif (Control(2) = '0') and (Control(1) = '0') and (Control(0) = '1') then
            if rising_edge(Slow) then
                n := n + 1;
                if (n >= 3) then
                    n := 0;
                    x := not x;
                end if;
            end if;
            Q <= (others => x);
        end if;
    end if;
end process;
end Attempting2;

```

2. Filter

Until the buffer is filled with data, the display shows 0. After the buffer is filled, at the arrival of a new data, the last number is dropped, all of the numbers are shifted right with one position and the new data is stored in the first position.

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity Filter is
    port ( Clk: in STD_LOGIC;
           ResetF, ResetD: in STD_LOGIC;
           Length: in STD_LOGIC_VECTOR (2 downto 0);
           Control: in STD_LOGIC_VECTOR (2 downto 0);
           Anodes: out STD_LOGIC_VECTOR (0 to 3);
           NumberOut: out STD_LOGIC_VECTOR (0 to 7));
end Filter;

architecture Attempting1 of Filter is
    component DataGenerator is

```



```

        port ( Clk, Reset : in STD_LOGIC;
              Control : in STD_LOGIC_VECTOR (2 downto 0);
              Q : inout STD_LOGIC_VECTOR (7 downto 0));
    end component;

    component ShiftRegister is
        port ( Clk : in STD_LOGIC;
              Output : out STD_LOGIC_VECTOR (3 downto 0) := "1110");
    end component;

    component FrequencyDivider is
        port( Clk : in STD_LOGIC;
              Reset : in STD_LOGIC;
              Q : out STD_LOGIC);
    end component;

    component AnodDeviver is
        port(Clk, Reset : in STD_LOGIC;
              Q : out STD_LOGIC);
    end component;

    signal Data : STD_LOGIC_VECTOR (7 downto 0);
    signal Sum : STD_LOGIC_VECTOR (11 downto 0) := "0000000000000";
    signal AverageTemp : STD_LOGIC_VECTOR (7 downto 0) := "00000000";
    signal ClkSlow: STD_LOGIC := '0';
    signal ClkAnod: STD_LOGIC := '0';
    signal Anod: STD_LOGIC_VECTOR (3 downto 0) := "1110";
    begin
        FreqDiv : FrequencyDivider port map(Clk => Clk,Reset => '1',Q => ClkSlow);
        NrGen : DataGenerator port map(Clk => Clk,Reset => ResetD,Control =>
Control,Q => Data);
        Anods : AnodDeviver port map(Clk => Clk,Reset => '1',Q => ClkAnod);
        Shifter : ShiftRegister port map( Clk => ClkAnod,Output => Anod);
        Anodes <= Anod;
        NumberOut(7) <= '1';

        process(ClkSlow, Length, ResetF)
            type ArrayNr is array (0 to 15) of STD_LOGIC_VECTOR (7 downto 0);
            variable Queue: ArrayNr := (others => "00000000");
            begin
                if ResetF = '1' then
                    Queue := (others => "00000000");
                    Sum <= "0000000000000";
                    AverageTemp <= "00000000";
                else
                    if (ClkSlow'Event) and (ClkSlow = '1') then

```

```

if Length = "000" then
elseif Length = "100" then
    Queue(15) := Queue(14);
    Queue(14) := Queue(13);
    Queue(13) := Queue(12);
    Queue(12) := Queue(11);
    Queue(11) := Queue(10);
    Queue(10) := Queue(9);
    Queue(9) := Queue(8);
    Queue(8) := Queue(7);
    Queue(7) := Queue(6);
    Queue(6) := Queue(5);
    Queue(5) := Queue(4);
    Queue(4) := Queue(3);
    Queue(3) := Queue(2);
    Queue(2) := Queue(1);
    Queue(1) := Queue(0);
    Queue(0) := Data;
    Sum <= Queue(0) + Queue(1);
    AverageTemp <= Sum (8 downto 1);
elseif Length = "101" then
    Queue(15) := Queue(14);
    Queue(14) := Queue(13);
    Queue(13) := Queue(12);
    Queue(12) := Queue(11);
    Queue(11) := Queue(10);
    Queue(10) := Queue(9);
    Queue(9) := Queue(8);
    Queue(8) := Queue(7);
    Queue(7) := Queue(6);
    Queue(6) := Queue(5);
    Queue(5) := Queue(4);
    Queue(4) := Queue(3);
    Queue(3) := Queue(2);
    Queue(2) := Queue(1);
    Queue(1) := Queue(0);
    Queue(0) := Data;
    Sum <= Queue(0) + Queue(1) + Queue(2) +
    Queue(3);
    AverageTemp <= Sum (9 downto 2);
elseif Length = "110" then
    Queue(0) := Data;
    Queue(15) := Queue(14);
    Queue(14) := Queue(13);
    Queue(13) := Queue(12);
    Queue(12) := Queue(11);

```

```

Queue(11) := Queue(10);
Queue(10) := Queue(9);
Queue(9) := Queue(8);
Queue(8) := Queue(7);
Queue(7) := Queue(6);
Queue(6) := Queue(5);
Queue(5) := Queue(4);
Queue(4) := Queue(3);
Queue(3) := Queue(2);
Queue(2) := Queue(1);
Queue(1) := Queue(0);
Queue(0) := Data;
Sum <= Queue(0) + Queue(1) + Queue(2) +
Queue(3) + Queue(4) + Queue(5) + Queue(6) + Queue(7);
AverageTemp <= Sum (10 downto 3);
elsif Length = "111" then
Queue(0) := Data;
Queue(15) := Queue(14);
Queue(14) := Queue(13);
Queue(13) := Queue(12);
Queue(12) := Queue(11);
Queue(11) := Queue(10);
Queue(10) := Queue(9);
Queue(9) := Queue(8);
Queue(8) := Queue(7);
Queue(7) := Queue(6);
Queue(6) := Queue(5);
Queue(5) := Queue(4);
Queue(4) := Queue(3);
Queue(3) := Queue(2);
Queue(2) := Queue(1);
Queue(1) := Queue(0);
Queue(0) := Data;
Sum <= Queue(0) + Queue(1) + Queue(2)
+Queue(3) + Queue(4) + Queue(5) + Queue(6) + Queue(7) +Queue(8) + Queue(9) + Queue(10)
+Queue(11) + Queue(12) + Queue(13) +Queue(14);
AverageTemp <= Sum (11 downto 4);
end if;
end if;
end if;
end process;

process(ClkAnod)
variable State : integer := 0;
begin
if Anod = "1110" then

```

```

case AverageTemp(3 downto 0) is
    when "0000" => NumberOut (0 to 6) <= "0000001";
    when "0001" => NumberOut (0 to 6) <= "1001111";
    when "0010" => NumberOut (0 to 6) <= "0010010";
    when "0011" => NumberOut (0 to 6) <= "0000110";
    when "0100" => NumberOut (0 to 6) <= "1001100";
    when "0101" => NumberOut (0 to 6) <= "0100100";
    when "0110" => NumberOut (0 to 6) <= "0100000";
    when "0111" => NumberOut (0 to 6) <= "0001111";
    when "1000" => NumberOut (0 to 6) <= "0000000";
    when "1001" => NumberOut (0 to 6) <= "0000100";
    when "1010" => NumberOut (0 to 6) <= "0001000";
    when "1011" => NumberOut (0 to 6) <= "1100000";
    when "1100" => NumberOut (0 to 6) <= "0110001";
    when "1101" => NumberOut (0 to 6) <= "1000010";
    when "1110" => NumberOut (0 to 6) <= "0110000";
    when "1111" => NumberOut (0 to 6) <= "0111000";
    when others => NumberOut (0 to 6) <= "1000001";
end case;
elsif Anod = "1101" then
    case AverageTemp(7 downto 4) is
        when "0000" => NumberOut (0 to 6) <= "0000001";
        when "0001" => NumberOut (0 to 6) <= "1001111";
        when "0010" => NumberOut (0 to 6) <= "0010010";
        when "0011" => NumberOut (0 to 6) <= "0000110";
        when "0100" => NumberOut (0 to 6) <= "1001100";
        when "0101" => NumberOut (0 to 6) <= "0100100";
        when "0110" => NumberOut (0 to 6) <= "0100000";
        when "0111" => NumberOut (0 to 6) <= "0001111";
        when "1000" => NumberOut (0 to 6) <= "0000000";
        when "1001" => NumberOut (0 to 6) <= "0000100";
        when "1010" => NumberOut (0 to 6) <= "0001000";
        when "1011" => NumberOut (0 to 6) <= "1100000";
        when "1100" => NumberOut (0 to 6) <= "0110001";
        when "1101" => NumberOut (0 to 6) <= "1000010";
        when "1110" => NumberOut (0 to 6) <= "0110000";
        when "1111" => NumberOut (0 to 6) <= "0111000";
        when others => NumberOut (0 to 6) <= "1000001";
    end case;
elsif anod = "1011" then
    case Data(3 downto 0) is
        when "0000" => NumberOut (0 to 6) <= "0000001";
        when "0001" => NumberOut (0 to 6) <= "1001111";
        when "0010" => NumberOut (0 to 6) <= "0010010";
        when "0011" => NumberOut (0 to 6) <= "0000110";
        when "0100" => NumberOut (0 to 6) <= "1001100";

```

```

        when "0101" => NumberOut (0 to 6) <= "0100100";
        when "0110" => NumberOut (0 to 6) <= "0100000";
        when "0111" => NumberOut (0 to 6) <= "0001111";
        when "1000" => NumberOut (0 to 6) <= "0000000";
        when "1001" => NumberOut (0 to 6) <= "0000100";
        when "1010" => NumberOut (0 to 6) <= "0001000";
        when "1011" => NumberOut (0 to 6) <= "1100000";
        when "1100" => NumberOut (0 to 6) <= "0110001";
        when "1101" => NumberOut (0 to 6) <= "1000010";
        when "1110" => NumberOut (0 to 6) <= "0110000";
        when "1111" => NumberOut (0 to 6) <= "0111000";
        when others => NumberOut (0 to 6) <= "1000001";
    end case;
elsif anod = "0111" then
    case Data(7 downto 4) is
        when "0000" => NumberOut (0 to 6) <= "0000001";
        when "0001" => NumberOut (0 to 6) <= "1001111";
        when "0010" => NumberOut (0 to 6) <= "0010010";
        when "0011" => NumberOut (0 to 6) <= "0000110";
        when "0100" => NumberOut (0 to 6) <= "1001100";
        when "0101" => NumberOut (0 to 6) <= "0100100";
        when "0110" => NumberOut (0 to 6) <= "0100000";
        when "0111" => NumberOut (0 to 6) <= "0001111";
        when "1000" => NumberOut (0 to 6) <= "0000000";
        when "1001" => NumberOut (0 to 6) <= "0000100";
        when "1010" => NumberOut (0 to 6) <= "0001000";
        when "1011" => NumberOut (0 to 6) <= "1100000";
        when "1100" => NumberOut (0 to 6) <= "0110001";
        when "1101" => NumberOut (0 to 6) <= "1000010";
        when "1110" => NumberOut (0 to 6) <= "0110000";
        when "1111" => NumberOut (0 to 6) <= "0111000";
        when others => NumberOut (0 to 6) <= "1000001";
    end case;
else
    NumberOut (0 to 6) <= "1000001";
end if;

end process;
end Attempting1;

```

3. Shift register

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

```

```

entity ShiftRegister is
    port ( Clk : in STD_LOGIC;
           Output : out STD_LOGIC_VECTOR (3 downto 0) );
end ShiftRegister;

architecture SRegister of ShiftRegister is
    signal N: STD_LOGIC_VECTOR (3 downto 0) := "1110";
begin
    process(Clk)
    begin
        if rising_edge(Clk) then
            case N is
                when "1110" => Output <= "1101";
                                N <= "1101";
                when "1101" => Output <= "1011";
                                N <= "1011";
                when "1011" => Output <= "0111";
                                N <= "0111";
                when others => Output <= "1110";
                                N <= "1110";
            end case;
        end if;
    end process;
end architecture SRegister;

```

4. Frequency Divider

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

entity FrequencyDivider is
    port( Clk, Reset : in STD_LOGIC;
          Q : out STD_LOGIC);
end entity;

architecture Flash of FrequencyDivider is
begin
    process(CLK,Reset)
        variable QINT : STD_LOGIC_VECTOR (24 downto 0) := (others => '0');
        variable N : STD_LOGIC := '0';
    begin
        if Reset = '1' then
            if (Clk'event) and (Clk ='1') then

```

```

        QInt := QInt + 1;
        if QInt = "1011111010111100001000000" then
            Q <= N;
            QInt := (others => '0');
            N := not N;
        end if;
    end if;
else
    Q <= '0';
end if;
end process;
end Flash;

```

5. Frequency Divider for the Anode

```

library IEEE;
use IEEE.STD_LOGIC_1164.ALL;
use IEEE.STD_LOGIC_ARITH.ALL;
use IEEE.STD_LOGIC_UNSIGNED.ALL;

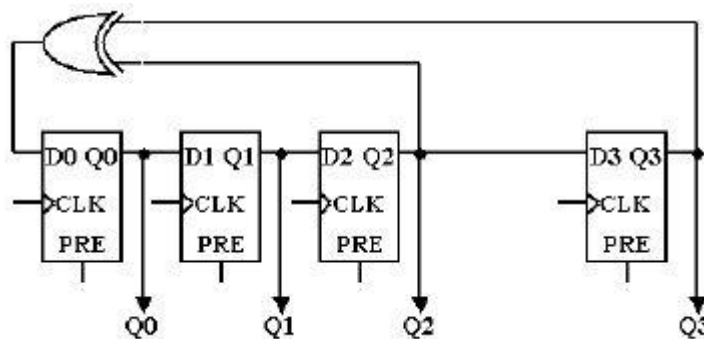
entity AnodDevier is
    port(Clk, Reset : in STD_LOGIC;
         Q : out STD_LOGIC);
end entity;

architecture Flash of AnodDevier is
begin
    process(Clk, Reset)
        variable QInt : STD_LOGIC_VECTOR (14 downto 0) := (others => '0');
        variable N : STD_LOGIC := '0';
    begin
        if Reset = '1' then
            if (Clk'event) and (Clk = '1') then
                QInt := QInt + 1;
                if QInt >= "110000110101000" then
                    Q <= N;
                    QInt := (others => '0');
                    N := not N;
                end if;
            end if;
        else
            Q <= '0';
        end if;
    end process;
end Flash;

```

VI. Inputs. Outputs. Internal signals.

For the pseudo random number generator the following signals were used: Clk, Reset and Control as input signals and Q as an input-output signal. The signal Q is an input signal for the number generator, but it is also an input for the buffer. There is also an internal signal called Slow, which receives the output signal from the frequency divider. Clk represents the clock signal. Control is a vector of 3 elements and it stores the control option chosen from the control switches. Q represents the 7 bit output of the number generator, which will then become an input signal for the filter.



We also used some internal variables, similar to the inputs and outputs of a D flip-flop, the circuit that stands at the base of this component which generates the sequence of pseudo random numbers. The solution chosen for generating these sequences can also be seen in the figure above. These numbers are visible on the display at a time interval of one second, due to the fact that a frequency divider was also used.

For the Filter we used the following signals: Clk, ResetF, ResetD, Length and Control as input signals and Anodes and NumberOut as outputs. Clk represents the clock signal of one second created with the help of the Frequency Divider. Reset is the signal which will reset the entire system, bringing it back to the initial value 0. The signal Length has the role of storing the data given by the user with the help of the Length Control switches and it represents the number of numbers generated and also the length of the buffer. Control is the signal received from the Control switches which are also at the user's disposal making it possible to choose from the different options presented in the table on page 3. Anodes is the signal which selects the anode on which the number will be displayed. The numbers will be displayed in hexadecimal, but

before displaying them, a link to the Shift Register component will be made, with the purpose of making the numbers appear on the 4 displays in real time, even though this cannot happen simultaneously. NumberOut represents a vector which defines the 7 segments which compose a display. Their significance is shown in the following picture.



We also used the following internal signals: Data, Sum, AverageTemp, ClkSlow, ClkAnod and Anod. Data represent the number received from the output of the Pseudo Random Number Generator. Sum stores the sum until present time and AverageTemp is a signal which stores the temporary average. ClkSlow is the one second clock signal received from the Frequency Divider and ClkAnod is the clock signal for the anode made with the help of the Frequency Divider for the Anode component. Anode represents the anode selected for displaying the data, with the help of the Shift Register component.

For the Shift Register we used as input the Clk signal and as output the signal Output. Clk represents the clock signal present in the sensitivity list of the process that it is a part of. The result of the shifting will be sent on the Output signal. Besides these two signals, another internal signal was also used to store the previous state of the register. This component is used in order to make the apparition of the numbers on the display simultaneous. As the usage of the 4 displays simultaneously is impossible, this is done with the help of this component. Everything happens at a very high frequency, so that this shift, which happens very rapidly, is unnoticeable.

For the Frequency Divider we used the following signals: Clk and Reset as input signals and Q as the output. Clk is the clock signal, Reset has the role of resetting the divider and Q is the output provided at the end of the counting process. This component has the role of bringing the process of displaying the numbers and the sistem's clock down to one second.

For the Frequency Divider for the Anode the same signals were used as in the case of the previous Frequency Divider: Clk and Reset as input signals and Q as the output. This component makes the numbers on the display appear in “real time”.

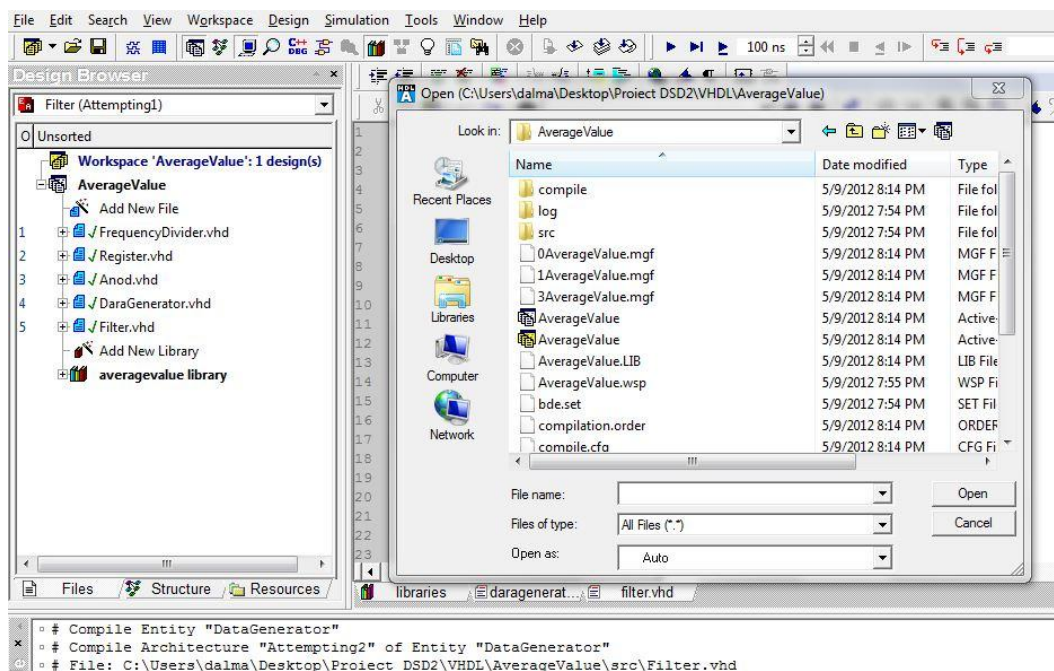
VII. Justification of the Chosen Solution

For this project’s implementation we chose five components in order to better illustrate the functionality of this system. These five components were presented in the previous chapters, and the connections between these components can be seen on the block scheme shown on page 4. Given the chosen solution, the interconnection of these components was done relatively easily which also makes it a lot easier for a less experimented user to understand the system’s functionality by taking a look at the block scheme or from the source code. In this solution that we chose, the processes and the sensitivity lists play a crucial role in defining the overall system. The interconnection of all these compnents was done in the process of the Filter component.

VIII. Users’s Manual

1. Active HDL

The user runs the Active HDL program. Then we choose from the File menu the option Open and from the new window which opens we can browse the computer in order to find the project from the directory it was saved in. Then the user opens “AverageValue.aws”.



On the left side of the screen, in “Design Browser” the user can see all the components of the current project. In our case we have: DataGenerator, Filter, Register, FrequencyDivider and Anod, all of them having the extension .vhd. By selecting one of these components, the user will be able to see the source code of that component.

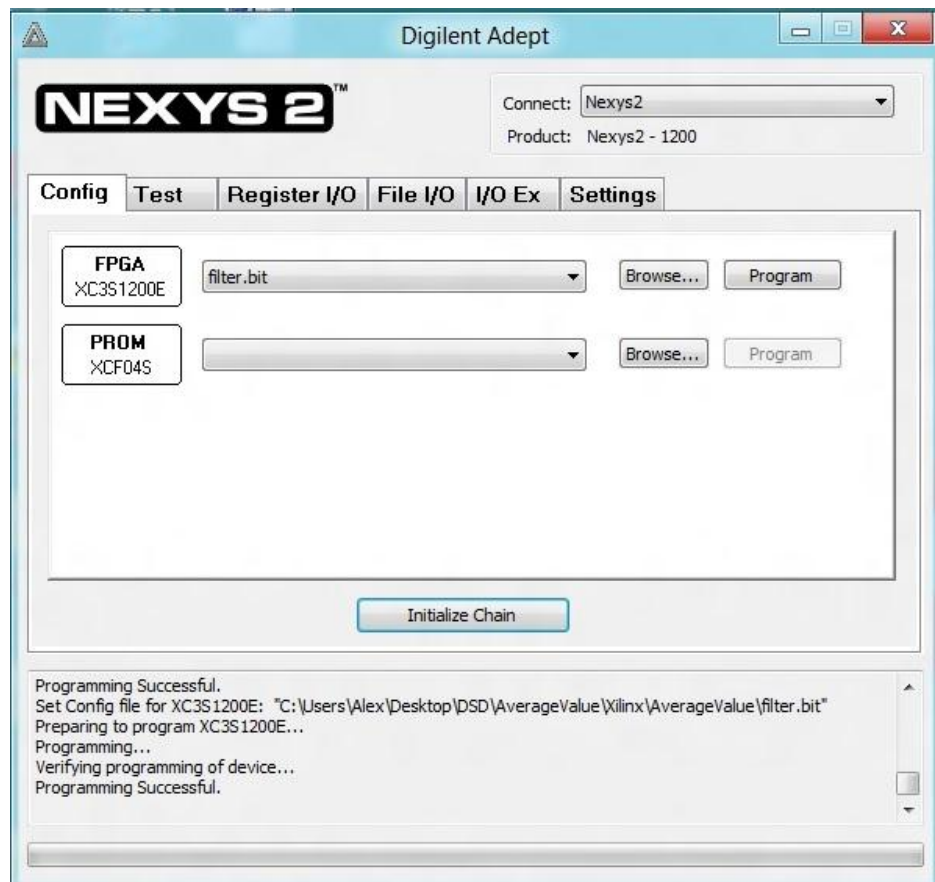
2. Xilinx ISE and the FPGA Board

First of all the user will have to load the VHDL source codes with the help of the option Add Source. All files with the extension .src should be loaded at this stage. This enables the source code written in VHDL to be opened with the help of Xilinx. On the left side of the screen, from the Processes window, we have to choose the option User Constraints-Assign Package Pins in order to select the input and output pins with respect to the pin’s names present on the FPGA board. In our case:

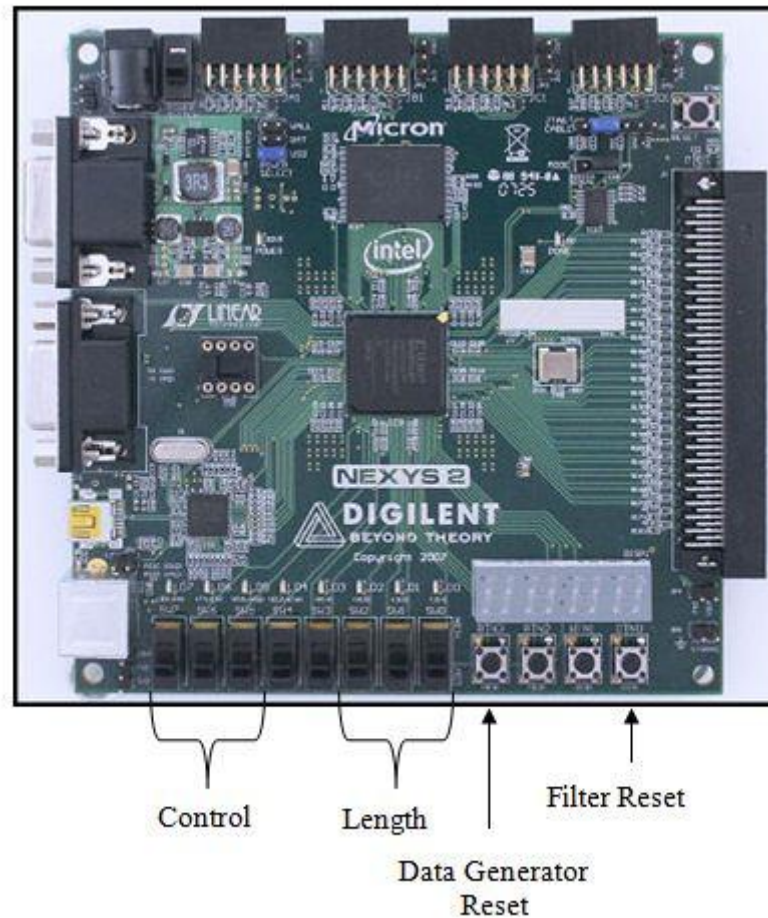
```
NET "Clk" LOC = "B8";
NET "Resetf" LOC = "B18" ;
NET "Resetd" LOC = "H13" ;
NET "Length<0>" LOC = "G18" ;
NET "Length<1>" LOC = "H18" ;
NET "Length<2>" LOC = "K18" ;
NET "Control<0>" LOC = "L13" ;
NET "Control<1>" LOC = "N17" ;
NET "Control<2>" LOC = "R17" ;
NET "Anodes<3>" LOC = "F17" ;
NET "Anodes<2>" LOC = "H17" ;
NET "Anodes<1>" LOC = "C18" ;
NET "Anodes<0>" LOC = "F15" ;
NET "NumberOut<0>" LOC = "L18";
NET "NumberOut<1>" LOC = "F18";
NET "NumberOut<2>" LOC = "D17";
NET "NumberOut<3>" LOC = "D16";
NET "NumberOut<4>" LOC = "G14";
```

```
NET "NumberOut<5>" LOC = "J17";  
NET "NumberOut<6>" LOC = "H14";  
NET "NumberOut<7>" LOC = "C17";
```

After this, we will choose Generate Programming File. Now the project is ready to be loaded on the FPGA board. The user runs the Digilent ExPort program. First we initialize the board using the option “Initialize Chain” and then we choose the program we wish to load on the board (it has the extension .bit). Then we push the Program Chain button which will become active.



Now all that's left to do is to test the project on the FPGA board using the assigned switches and buttons.



IX. Possibilities of Further Development and Improvement

A first improvement which could be brought to this system could be a more exact approximation of the average value of the numbers: those with decimal values larger or equal to 50 should be approximated by adding to the next number, and those smaller than 50 should be approximated by lack to the previous number. In this way the accuracy of the computed average will be higher.

Another improvement which could be made in the future is to give the user the opportunity to choose from a larger variety of options. One of these could be the possibility of having numbers larger than 8 bits. In this way we could extend the computing system to numbers larger than 255, increasing its applicability.